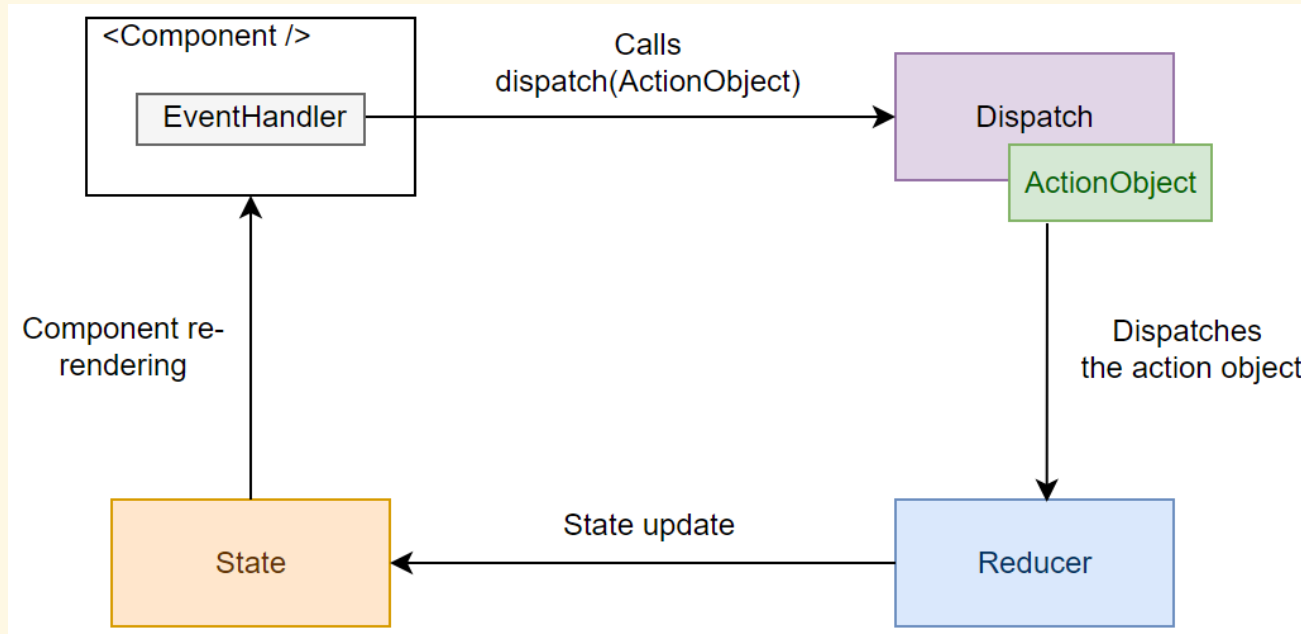

Reducer, Redux

Reducer

Introduction

- ▶ In React, the state management logic takes part in the component body
 - ▶ Bad example of the “separation of concerns” principle
 - ▶ Mix of state management and rendering logic
 - ▶ The state transition is unclear and not predictable, as it’s possible to update the state directly from anywhere in the component
- ➔ Difficult to write, read, maintain and test!
- ▶ `useReducer()`
 - ▶ An alternative hook to `useState()`
 - ▶ Helps to extract the state management out of the component
 - ▶ More helpful when the state is complex and composed of multiple properties

Principle



- ▶ The event handler does not update the state directly, but dispatches an action
- ▶ React redirects the action to the reducer
- ▶ Depending on the action, the reducer updates the state (in an immutable manner)
- ▶ The state update will cause the component to re-render

Usage

- ▶ `import { useReducer } from 'react';`
- ▶ `const [state, dispatch] = useReducer(reducer, initState);`
 - ▶ `initState`: initial value of the state
 - ▶ `state`: current state value
 - ▶ `reducer`: a user-provided function that takes the current state, an **action object** then evaluates and returns a new state
 - ▶ `dispatch`: a function that dispatches an action object
- ▶ An **action object** normally includes:
 - ▶ `Type`: what the action is
 - ▶ `Payload`: additional information to the action

Simple Example: The Dispatcher

```
import { useReducer } from 'react';

function Regulator() {
  const [state, dispatch] = useReducer(RegulatorReducer, 10);

  return <View>
    <Text>Value: {state}</Text>

    <Button title="+" onPress={
      () => dispatch({ type: 'increase', amount: 1 }) } />
    <Button title="-" onPress={
      () => dispatch({ type: 'decrease', amount: 1 }) } />
    <Button title="++" onPress={
      () => dispatch({ type: 'increase', amount: 10 }) } />
    <Button title="--" onPress={
      () => dispatch({ type: 'decrease', amount: 10 }) } />
    <Button title="Reset" onPress={
      () => dispatch({ type: 'reset' }) } />
  </View>
}
```

Simple Example: The Reducer

```
function RegulatorReducer(state, action) {  
  let newState;  
  
  switch (action.type) {  
    case 'reset':  
      newState = 0;  
      break;  
    case 'increase':  
      newState = state + action.amount;  
      break;  
    case 'decrease':  
      newState = state - action.amout;  
      break;  
    default:  
      throw new Error();  
  }  
  
  return newState;  
}
```

Other Remarks

- ▶ It's a good practice to combine all the component's state data into one state object and manage it with the reducer
- ▶ The state can be lazily initialized with the following syntax:
 - ▶ `const [state, dispatch] = useReducer(reducer, initArg, initFunc);`
 - ▶ The initial state will be set to `initFunc(initArg)` where `initFunc()` is a user-defined function
 - ▶ This helps to further extract the state initialization out of the component

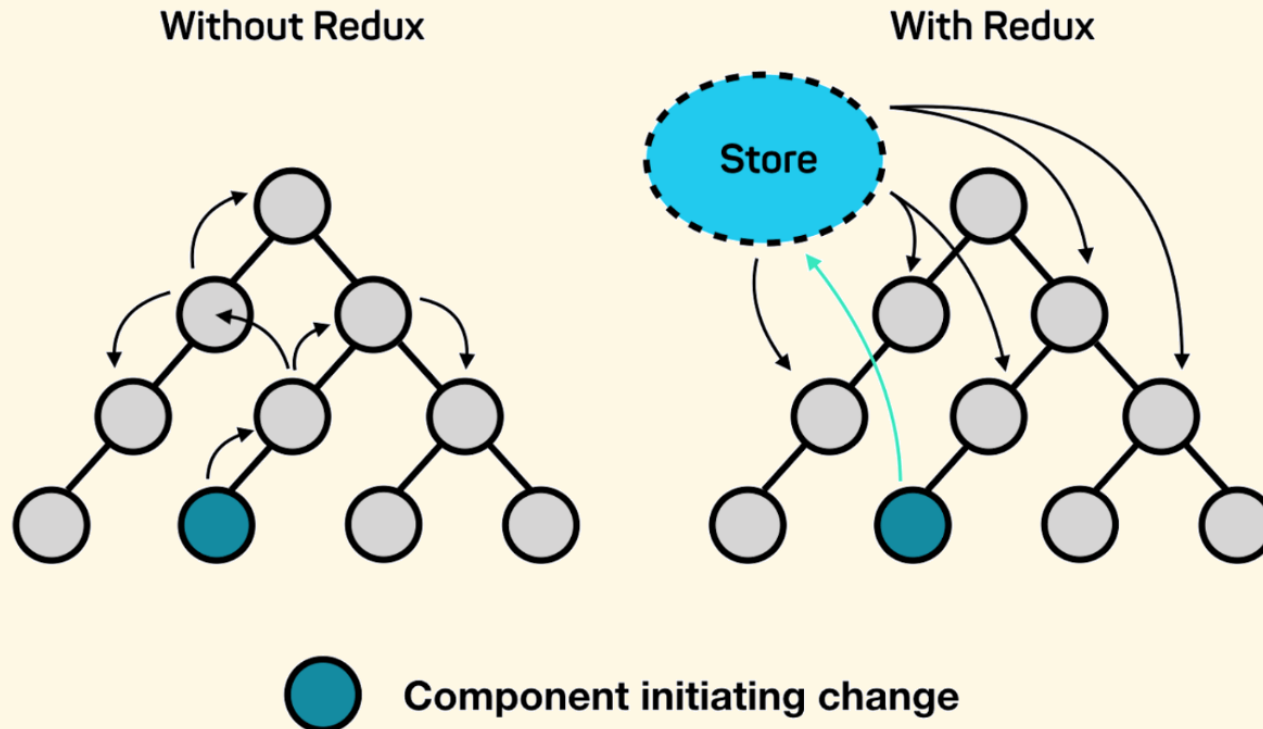
Exercise

- ▶ Rewrite the BMI calculator app using React reducer.
 - ▶ Make sure to define all information in a single state object.

Redux

Introduction

- ▶ Managing components' state in React is painful when the DOM is complex (“prop drilling” problem)
- ▶ Redux comes to help!

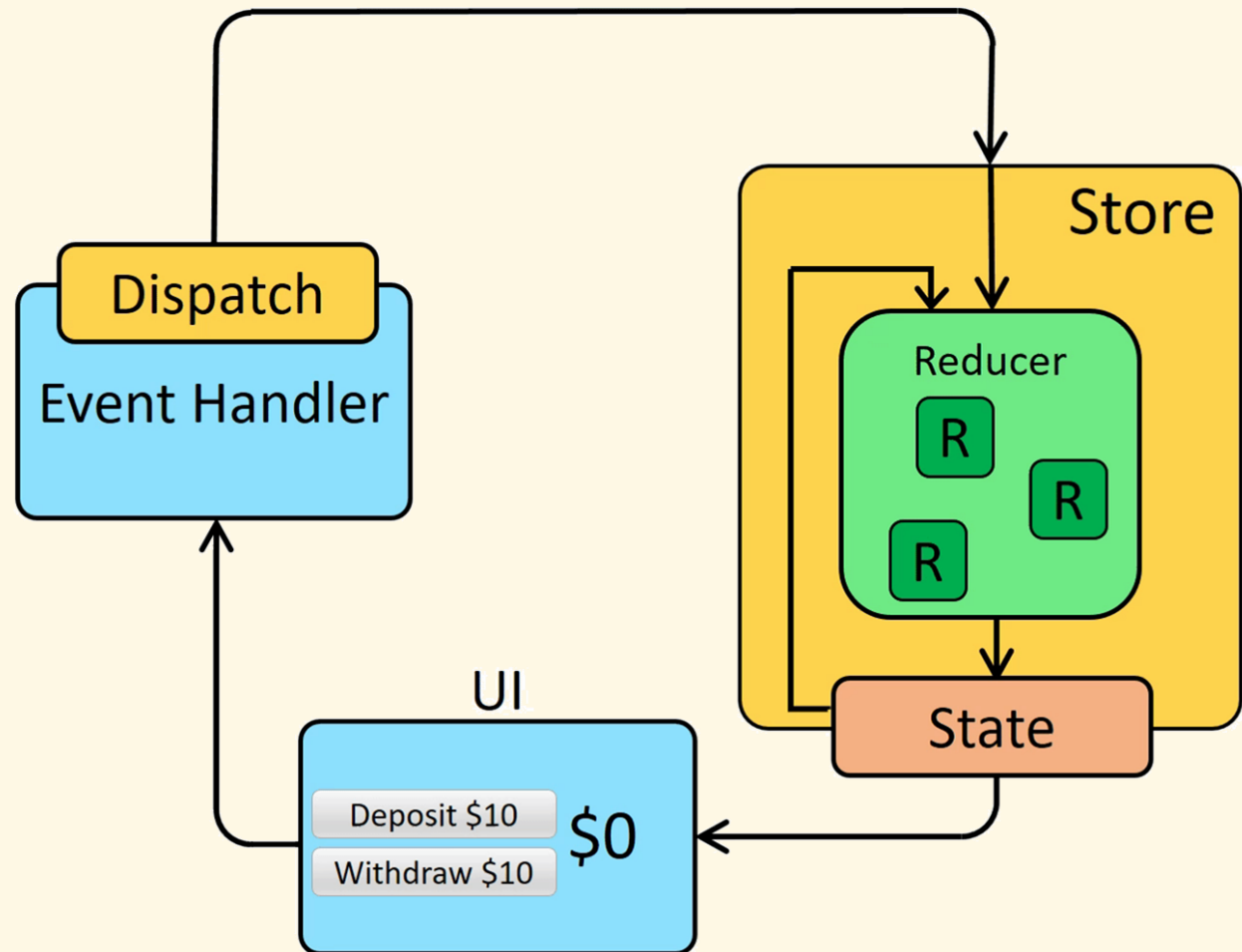


Redux

- ▶ 3rd-party library for predictable state container for JavaScript apps, especially React apps
- ▶ Alternative to React `useReducer()` and `useContext()`
- ▶ More suitable for large and complex apps
- ▶ Install:
 - ▶ `npm install react-redux @reduxjs/toolkit`
 - ▶ React Redux: React UI bindings layer for Redux
 - ▶ React Toolkit: package to simplify the Redux logic

Principle

- ▶ Very analogous to React reducer
- ▶ Store: a place to hold and update the state data
 - ▶ Single source of truth



Example: Reducer

```
import { createSlice } from '@reduxjs/toolkit';

export const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => { state.value += 1 },
    decrement: (state) => { state.value -= 1 },
    incrementByAmount:
      (state, action) => { state.value += action.payload }
  }
});

// Action creators are generated for each case reducer function
export const { increment, decrement, incrementByAmount }
  = counterSlice.actions;

export default counterSlice.reducer;
```

Redux requires that we write all state updates immutably, but Redux Toolkit's `createSlice()` and `createReducer()` APIs use Immer inside to allow to write “mutating” update logic that becomes correct immutable updates

Example: Store

- ▶ Add the counter slice to the store to tell the store to use this slice reducer function to handle all updates to that state

```
▶ import { configureStore }  
  from '@reduxjs/toolkit';  
import counterReducer  
  from '../features/counter/counterSlice';
```

```
export default configureStore({  
  reducer: {  
    counter: counterReducer,  
    another: anotherReducer,  
    yetAnother: yetAnotherReducer  
  }  
});
```

Example: Provider

```
import store from './app/store';  
import { Provider } from 'react-redux';  
  
export default function App() {  
  return <Provider store={store}>  
    // ...  
  </Provider>  
}
```


Example: Dispatching Actions

```
import { useSelector, useDispatch } from 'react-redux';
import { decrement, increment } from '../counterSlice';

function Counter() {
  const count = useSelector(state => state.counter.value);
  const dispatch = useDispatch();

  return <View>
    <Text>Value: {count}</Text>

    <Button title="+" onPress={
      () => dispatch(increment()) } />
    <Button title="-" onPress={
      () => dispatch(decrement()) } />
  </View>
}
```

Exercise

- ▶ Rewrite the BMI calculator app using Redux.